



Mobile Dev AI Team

Usage Guide & GitHub Documentation

Flutter | Android | iOS | Claude Code Subagents

Prepared by

Dinesh Wayaman

May 2026

1. What Is This?

Mobile Dev AI Team is an open-source multi-agent system for Claude Code that gives your mobile project a full AI development team — 8 specialized agents that discuss, review, challenge each other, and make decisions together before and after every feature is built.

Instead of asking Claude Code to do everything in one giant prompt, each agent has a specific role, expertise, and responsibilities. They work as a real team.

The 8 Agents



Project Manager

Breaks features into tasks, writes acceptance criteria, tracks progress

When: Start of every feature — before anything else



Architect

System design, pattern selection, tech decisions, code structure review

When: Pre-dev planning + post-dev code review



Senior Mobile Engineer

Flutter/Android/iOS implementation — writes clean, tested, production code

When: After Judge approves the architecture plan



QA Engineer

Test plans, unit/widget/integration tests, quality gates, coverage reports

When: Pre-dev (test plan) + post-dev (validation)



Security Expert

OWASP Mobile Top 10, threat modeling, auth review, data storage audit

When: Pre-dev design review + post-dev audit



UI/UX Expert

Material 3, Apple HIG, accessibility (WCAG 2.1 AA), all UI states

When: Any feature with screen or flow changes



Performance Analyzer

Frame rates, memory leaks, widget rebuilds, battery drain, jank

When: Post-dev + on-demand profiling



Judge

Final GO/HOLD/SHIP decisions, conflict resolution between agents

When: After pre-dev discussion + before shipping

2. Installation & Setup

Step 1 — Install Claude Code

```
npm install -g @anthropic-ai/claude-code
```

Step 2 — Add agents to your project

Option A: Clone the repo as a standalone project

```
git clone https://github.com/DineshWayaman/Agentic-Mobile-Dev-Team.git
cd Agentic-Mobile-Dev-Team
claude
```

Option B: Copy into an existing Flutter project

```
cp -r mobile-dev-agents/.claude your-flutter-project/
cp mobile-dev-agents/CLAUDE.md your-flutter-project/
cp -r mobile-dev-agents/decisions your-flutter-project/

cd your-flutter-project
claude
```

3. How to Use the Agents

3.1 Slash Commands (Recommended)

Slash commands trigger the right set of agents automatically. Type them directly in the Claude Code terminal.

Command	What it does	Agents involved
<code>/feature [description]</code>	Full pre-dev team discussion for a new feature	All 8 agents
<code>/review</code>	Post-dev team audit of what was just built	QA + Security + UI/UX + Performance + Architect + Judge
<code>/audit [area]</code>	Deep security + performance scan	Security + Performance + Judge
<code>/design-review</code>	UI/UX and architecture pass on a screen	UI/UX + Architect + Judge

<code>/ship [version]</code>	Final release check before publishing	PM + QA + Security + Judge
------------------------------	---------------------------------------	----------------------------

3.2 Call a Single Agent Directly

When you only need one specialist, call them by name:

```
Use the security-expert agent to audit my login code
Use the architect agent to design the data layer for offline sync
Use the performance-analyzer agent to check my home screen
Use the ui-ux-expert agent to review my cart screen
```

3.3 Just Talk Normally

Claude Code reads CLAUDE.md and knows when to bring agents in automatically. If you describe a feature, it triggers the pre-dev workflow before writing any code:

```
I need to build a notification system
Help me add offline sync to the orders screen
Build a biometric login feature
```

4. The Team Workflow

Every feature goes through two gates. Nothing is built until Gate 1 passes. Nothing ships until Gate 2 passes.

GATE 1 — PRE-DEVELOPMENT (before any code)

1. Project Manager → Breaks down feature, writes acceptance criteria
2. Architect → Proposes technical approach and file structure
3. Security Expert → Reviews design for threats, flags concerns
4. UI/UX Expert → Reviews screens, defines all UI states
5. QA Engineer → Defines test plan before code is written
6. Judge → Reviews all input → issues GO or HOLD

-  GO → Mobile Engineer starts implementing
-  HOLD → Blockers must be resolved first

DEVELOPMENT

7. Mobile Engineer → Implements feature following agreed architecture
8. Performance Analyzer → Watches for issues as code is written

GATE 2 — POST-DEVELOPMENT (before shipping)

9. QA Engineer → Runs tests, checks coverage, produces report
10. Security Expert → Audits implementation against OWASP Mobile Top 10
11. UI/UX Expert → Reviews final screens, checks accessibility
12. Performance Analyzer → Profiles for jank, memory leaks, battery drain
13. Architect → Reviews code structure and architecture compliance
14. Judge → Issues final SHIP / HOLD / BLOCK verdict

-  SHIP → Feature is cleared for release
-  HOLD → Specific fixes needed — returned to Mobile Engineer
-  BLOCK → Critical issue — must be resolved before anything continues

5. Real Example

Here's what happens when you type:

```
/feature Add biometric login to the app
```

Claude Code runs through the team in sequence:

```
[project-manager] Breaking down feature...
  Tasks: TASK-001 (Keystore/Keychain storage),
         TASK-002 (UI flow), TASK-003 (fallback PIN)

[architect] Proposing architecture...
  → Package: local_auth + flutter_secure_storage
  → Pattern: Clean arch – BiometricAuthService in auth/data/

[security-expert] Threat modeling...
  → M3 risk: fallback PIN can bypass biometric
  → Requirement: fallback must re-verify identity

[ui-ux-expert] Design review...
  → States: enrolled / not-enrolled / failed / locked-out

[qa-engineer] Test plan defined...
  → 4 unit tests, 2 widget tests, 1 integration test

[judge] Decision...
  → 🟢 GO – Condition: PIN fallback must re-authenticate

[mobile-engineer] Implementing...
  → Writing BiometricAuthService + UI + tests
```

6. Minimizing Token Usage

Running all 8 agents on every task is expensive. Use the right command for the task to avoid burning tokens on agents that aren't needed.

Use the right command for the task

Task type	What to use	Agents used
New feature	/feature	All 8
Bug fix	Talk to mobile-engineer directly	1
UI-only change	/design-review	3
Security concern	security-expert directly	1
Pre-release check	/ship	4

Judge fast-track rule

If all agents report no issues, the Judge responds in one line: "🚀 SHIP — all agents clear." Full verdicts only when there are actual conflicts or blockers.

7. File Structure Reference

```
your-project/
├── CLAUDE.md                # Orchestrator – team rules & workflow
├── decisions/
│   └── DECISIONS.md        # All team decisions logged here
├── .claude/
│   ├── agents/
│   │   ├── project-manager.md
│   │   ├── architect.md
│   │   ├── mobile-engineer.md
│   │   ├── qa-engineer.md
│   │   ├── security-expert.md
│   │   ├── ui-ux-expert.md
│   │   ├── performance-analyzer.md
│   │   └── judge.md
│   └── commands/
│       ├── feature.md
│       ├── review.md
│       ├── audit.md
│       └── design-review.md
```


└─ ship.md

8. Customizing Agents

Every agent is a plain Markdown file. Edit them to match your project:

- Change state management: Edit mobile-engineer.md — swap BLoC for Riverpod or Provider
- Change coverage targets: Edit the % in qa-engineer.md
- Add project conventions: Add a "Project Conventions" section to any agent file
- Add a new agent: Create a .md file in .claude/agents/ and reference it in CLAUDE.md
- Skip an agent: Remove it from the relevant slash command in .claude/commands/
- Per-agent model: Add model: claude-opus-4-6 to any agent frontmatter for heavier reasoning

9. Decisions Log

Every major decision the team makes is logged to decisions/DECISIONS.md automatically — a full audit trail of why things were built the way they were.

```
### 2026-05-31 Biometric Login – Architecture Decision
**Decision:** Use local_auth + flutter_secure_storage
**Rationale:** Handles both Android Keystore and iOS Keychain natively
**Alternatives rejected:** Custom implementation – too much risk surface
**Decided by:** Architect, confirmed by Judge
```